

Algorithm ?? vs. BFGS on the MGH Problems

1 The degeneracy at $k = 0$

Algorithm ?? minimizes $f(x) = \frac{1}{2}\|F(x)\|^2$, where $F = (f_1, \dots, f_m)^\top$ is the residual vector. At external iteration k it keeps one symmetric matrix H_i^k per residual component f_i , approximating $\nabla^2 f_i(x^k)$, and solves

$$M_k(s) = \sigma\|s\|^2 + \frac{1}{2} \sum_i q_i(s)^2, \quad q_i(s) = f_i(x^k) + \nabla f_i(x^k)^\top s + \frac{1}{2} s^\top H_i^k s, \quad (1)$$

accepting or rejecting $x^k + s$ by the ratio between actual and predicted reduction, and growing/shrinking σ accordingly. At $k = 0$, every $H_i^0 = 0$ (no secant pair has been observed yet), so (1) collapses to

$$M_0(s) = \sigma\|s\|^2 + \frac{1}{2} \sum_i (f_i(x^0) + \nabla f_i(x^0)^\top s)^2, \quad (2)$$

a single damped Gauss–Newton subproblem. Two things are worth noting about this particular instance of the subproblem. Unlike every subsequent iteration, $\sigma = \sigma_0 = 1$ here is a fixed constant, not yet informed by any accept/reject test on *this* problem, so how close (2)’s solution already sits to the scaled steepest-descent direction it approaches as $\sigma \rightarrow \infty$ (where the minimizer of (2) is asymptotically parallel to $-\nabla f(x^0)$) is essentially a guess about the local curvature scale of f at x^0 that the method has had no chance to calibrate yet. Moreover, there is, by construction, no curvature information at all to justify solving a quartic (rather than a quadratic) model here. The quartic term $\frac{1}{2} s^\top H_i^k s$ is identically zero for every i . So the very first step is the one iteration where the method’s own machinery (quasi-Newton curvature, calibrated regularization) has nothing to work with.

1.1 The closed form of M_0

Rather than appealing to a different algorithm, the cleanest justification comes directly out of (2) itself, which is an unconstrained, strictly convex quadratic in s and can be solved in closed form. Write $J := F'(x^0)$ for the Jacobian of F at x^0 (the matrix whose i -th row is $\nabla f_i(x^0)^\top$), so that $g := \nabla f(x^0) = J^\top F(x^0)$, and $A := J^\top J$ (symmetric positive semidefinite). Then

$$\nabla_s M_0(s) = (2\sigma I + A)s + g = 0 \implies s(\sigma) = -(A + 2\sigma I)^{-1}g.$$

Lemma 1.1.1 (Descent for every $\sigma > 0$). *For every $\sigma > 0$ and every $g \neq 0$, $s(\sigma)$ is a descent direction for f at x^0 , satisfying $g^\top s(\sigma) < 0$.*

Proof. Since $A + 2\sigma I \succ 0$ for $\sigma > 0$, its inverse is symmetric positive definite, hence

$$g^\top s(\sigma) = -g^\top (A + 2\sigma I)^{-1}g < 0 \quad \text{whenever } g \neq 0.$$

This completes the proof. □

So $s(\sigma)$ is a genuine descent direction for f for every $\sigma > 0$, not just asymptotically. The σ -loop never has a “wrong-direction” problem at $k = 0$. What it can have is a badly sized step, which is what the accept/reject test and the growth of σ are actually correcting for.

Proposition 1.1.2 (Alignment with $-g$ as $\sigma \rightarrow \infty$). *As $\sigma \rightarrow \infty$,*

$$s(\sigma) = -\frac{1}{2\sigma}g + o\left(\frac{1}{\sigma}\right) \implies \frac{s(\sigma)}{\|s(\sigma)\|} \longrightarrow -\frac{g}{\|g\|}. \quad (3)$$

Proof. Factor 2σ out of the inverse:

$$s(\sigma) = -(A + 2\sigma I)^{-1}g = -\frac{1}{2\sigma} \left(I + \frac{A}{2\sigma} \right)^{-1} g.$$

As $\sigma \rightarrow \infty$, $A/(2\sigma) \rightarrow 0$, and matrix inversion is continuous at the identity, so $(I + A/(2\sigma))^{-1} \rightarrow I$, which gives the stated expansion. This completes the proof. $\square$

In words, for σ large enough, the solution of the degenerate $k = 0$ subproblem (2) already is, to leading order, a steepest-descent step of length $\approx 1/2\sigma$. The two are asymptotically the same computation. This is a direct consequence of (2)’s own closed form, entirely internal to Algorithm ??.

In practice, the rejection ladder $\sigma_0, \sigma_{j+1} = \gamma\sigma_j$ ($\gamma > 1$ the growth factor) tries $s(\sigma_0), s(\sigma_1), \dots$ until one is accepted. By Proposition 1.1.2, once $\sigma_j \gg \|A\|/2$ the successive trials are already essentially parallel to $-g$ and shrink by the fixed factor $1/\gamma$, so once it reaches that scale, the ladder is already behaving like a backtracking line search along $-g$. This is also a reason not to let σ_0 run away unchecked. Because σ is only shrunk, never reset, between outer iterations, an unusually large σ_0 , forced whenever the true first step happens to be small, persists into σ_1 and can over-dampen the next iteration’s subproblem as well.

1.2 Why consider this scheme

Solving a coefficient-estimation problem for a PDE is expensive. Evaluating the objective function requires solving the PDE once, and differentiating that objective requires differentiating the PDE solver itself. In [5] we solved this problem without derivatives, but the resulting computational cost was prohibitively high. In this work we instead use derivatives obtained by automatic differentiation.

This approach computes an exact derivative of the PDE solver, rather than a finite-difference approximation, via forward-mode automatic differentiation, using Julia’s `ForwardDiff.jl`. We chose forward mode over reverse (backward) mode because reverse-mode differentiation must first record every elementary operation performed by the solver onto a computational tape before it can evaluate the derivative, and since the PDE solver performs a very large number of operations, this tape would require a prohibitive amount of memory, on the order of 3 gigabytes for a single derivative evaluation in our setting.

Using as much derivative information as possible is valuable in principle, but Figure 1 shows why this is impractical here. The cost of computing the exact Hessian by automatic differentiation grows far faster with the problem dimension than the cost of the gradient, so exact second-order information should be avoided, relying instead on first-order (gradient) information alone.

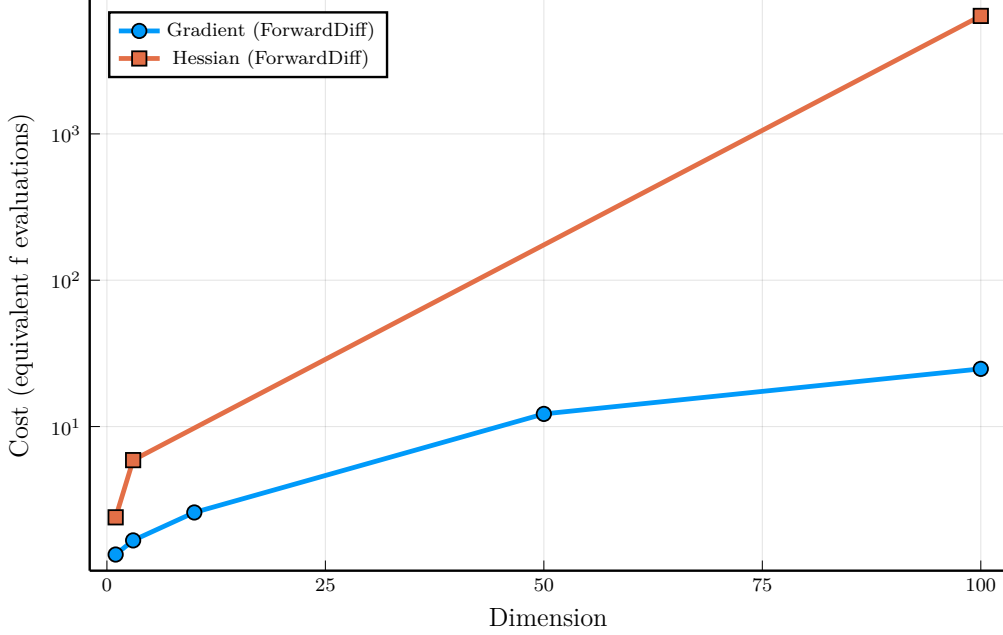


Figure 1: Cost, in equivalent evaluations of f , of computing the gradient and the Hessian of the PDE solver’s objective by automatic differentiation, as a function of the problem dimension.

At dimension 100, Figure 1 shows that a single gradient evaluation already costs about as much as 25 evaluations of f , while a single Hessian evaluation costs more than 6400 evaluations of f .

For this reason, we need to make full use of the information already available to us, particularly the gradient, in order to solve this problem efficiently.

2 Computational tests

In this section we evaluate the performance of Algorithm ?? and compare it against a classical sum-of-squares approach. We run two types of tests. In the first, we consider the MGH (Moré, Garbow, and Hillstom) sum-of-squares test problems [9], a benchmark of 34 classical problems. This part is important to us because it helps us settle on a good solver and Hessian approximation. In the second part, we solve a hydraulic problem in which we need to estimate a coefficient of a PDE called the Saint-Venant equation.

2.1 Algorithm ?? for MGH problems

Algorithm ?? minimizes $f(x) = \frac{1}{2}\|F(x)\|_2^2$ (Section 3, 1) by minimizing, at each iteration, a local quartic model of the residuals in place of a line search. Following Algorithm ?? (Eq. (21)), the step s at iteration k solves

$$M_k(s) = \sigma\|s\|_2^2 + \frac{1}{2} \sum_{i=1}^m q_i(s)^2, \quad q_i(s) = f_i(x_k) + \nabla f_i(x_k)^\top s + \frac{1}{2} s^\top H_i s,$$

where f_i and ∇f_i are the i -th residual and its gradient at x_k (Section 3), H_i is the current quasi-Newton approximation to its Hessian. Four further variants are compared in Table 1, and $\sigma\|s\|_2^2$ is the Levenberg-Marquardt-style regularization of Algorithm ?? that damps the step in place of a line search. The candidate $x_k + s$ is checked for feasibility as in Step 3 (Eq. ??). Acceptance then uses the ratio ρ_k between the actual and the model-predicted reduction in f , a refinement of Algorithm ??’s simpler decrease test (Eq. (25)), accepting whenever $\rho_k \geq \eta_1$ for a fixed threshold η_1 .

The implementation tested below uses the regularization update of Eq. (??) with the following parameters. The first iteration starts at $\sigma = 1$. Each accepted step divides σ by a fixed factor

$\sigma_{grow} = 10$, and each rejected step, as in Eq. (30), multiplies σ by the same factor $\sigma_{grow} = 10$ and resolves the subproblem at the same point, until a step is accepted. Between outer iterations, σ always carries over from the previous one rather than being reset. The floor $\sigma_{\min} = 10^{-8}$ and the ceiling $\sigma_{\max} = 10^{12}$ are used purely for numerical stability, preventing σ from underflowing or overflowing over the course of a run.

Every run reported below stops under one of three point-wise criteria, evaluated with the same default tolerances for Algorithm ?? and for Optim.jl’s BFGS [8] (kept equal so the comparison is fair):

- **Gradient:** the gradient norm reaches $\|\nabla f\| \leq 10^{-8}$. This is a genuine first-order stationary point (reported as `grad_conv` for both Algorithm ?? and BFGS in the tables below).
- **Step:** the last step has (numerically) zero length. The solver stalls and stops making progress (reported as `step_conv` for both Algorithm ?? and BFGS).
- **Function:** the objective value does not change between two consecutive iterations. The iterate is stuck on a plateau (reported as `func_conv` for both Algorithm ?? and BFGS).
- **Stalled:** Algorithm ??’s regularization parameter exceeded its ceiling, that is, $\sigma > 10^{12}$ (reported as `reg_stalled`. BFGS has no equivalent criterion).

Gradient, Step, and Function use the same name for both methods by construction: BFGS’s own status is classified by checking `Optim.g_converged`, then `Optim.x_converged`, then `Optim.f_converged` on its result, in that order, so the two methods can be compared under one shared vocabulary instead of each reporting its own native termination code.

Table 1 compares the six Algorithm ?? Hessian-update models, each run independently on the 34 MGH least-squares problems [9]. Its “Converged (gradient)” column counts how many of the 34 problems stopped under the gradient criterion. The remaining columns are mean $\pm$ standard deviation over the 34 problems. Each model keeps its own set of matrices $H_i \approx \nabla^2 f_i(x_k)$, $i = 1, \dots, m$, updated at every accepted step from $s = x_{k+1} - x_k$ and $y_i = \nabla f_i(x_{k+1}) - \nabla f_i(x_k)$:

- **BFGS:** the classical rank-two secant update (Eq. (19)),

$$H_i \leftarrow H_i - \frac{H_i s s^T H_i}{s^T H_i s} + \frac{y_i y_i^T}{s^T y_i}.$$

- **SR1:** the symmetric rank-one update (Eq. (20)),

$$H_i \leftarrow H_i + \frac{(y_i - H_i s)(y_i - H_i s)^T}{(y_i - H_i s)^T s}.$$

- **DW** (Dennis-Wolkowicz [3], Theorem 4.5(iii)): a Broyden-class update that adds a third, symmetrizing rank-one term to the BFGS pair,

$$H_i \leftarrow H_i - \frac{H_i s s^T H_i}{s^T H_i s} + \frac{y_i y_i^T}{s^T y_i} + (s^T y_i) w_i w_i^T, \quad w_i = \frac{y_i}{s^T y_i} - \frac{H_i s}{s^T H_i s}.$$

- **PSB** (Powell-Symmetric-Broyden [10]): a rank-two update that does not preserve positive definiteness (like SR1),

$$H_i \leftarrow H_i + \frac{(y_i - H_i s)s^T + s(y_i - H_i s)^T}{s^T s} - \frac{s^T (y_i - H_i s)}{(s^T s)^2} s s^T.$$

Table 1: Comparison of the Algorithm ?? Hessian-update models on the 34 MGH problems.

Model	Converged (gradient)	Iterations	Func. evals	Grad. evals	Time (s)
BFGS	29/34	56.88 ± 188.25	99.97 ± 351.85	57.88 ± 188.25	2.071 ± 6.121
SR1	32/34	23.12 ± 54.84	44.32 ± 112.92	24.12 ± 54.84	0.934 ± 2.992
DW	29/34	56.12 ± 181.91	103.24 ± 351.45	57.12 ± 181.91	1.865 ± 5.942
PSB	31/34	15.68 ± 22.53	31.38 ± 53.44	16.68 ± 22.53	0.424 ± 0.919

PSB and SR1 are the two best models. Together they solve the most problems under the gradient criterion, and PSB in particular does so with by far the fewest iterations, evaluations, and execution time on average (Table 1). SR1 nonetheless certifies gradient convergence on one more problem than PSB (32/34 against 31/34). To see exactly where they disagree, Table 2 isolates the three problems on which at least one of the two methods does not stop under the gradient criterion.

Table 2: Algorithm ?? (PSB) vs. Algorithm ?? (SR1): per-problem comparison on the MGH problems.

Problem	Method	f	$\ \nabla f\ $	Func. evals	Grad. evals	Criterion
MGH10	Algorithm ?? (PSB)	4.40×10^1	5.75×10^{-5}	299	127	step_conv
MGH10	Algorithm ?? (SR1)	4.40×10^1	5.50×10^{-2}	297	125	step_conv
MGH16	Algorithm ?? (PSB)	4.29×10^4	9.26×10^{-5}	98	36	func_conv
MGH16	Algorithm ?? (SR1)	4.29×10^4	6.26×10^{-5}	44	9	func_conv
MGH31	Algorithm ?? (PSB)	1.53	2.53×10^{-8}	62	21	func_conv
MGH31	Algorithm ?? (SR1)	1.53	4.39×10^{-9}	44	29	grad_conv

On MGH10 and MGH16, neither model reaches the gradient criterion, both stall under the step and function criteria, respectively, at essentially the same objective value, although Algorithm ?? with PSB reaches a smaller gradient norm on MGH10. For this reason, PSB is the best choice between the two approaches when weighing cost against benefit.

Table 3 repeats the comparison, but now running Algorithm ?? in full with PSB model on the same 34 problems. Newton/BFGS/BOBYQA solve only the small quartic model at each outer iteration, exactly as Algorithm ?? uses them, instead of standing in as the whole optimizer. “Newton” here is the exact Newton step on the quartic subproblem, computed via the Ipopt interior-point solver.

Function and gradient evaluations here count only the calls to the real residual and Jacobian made by Algorithm ??’s outer loop. The internal solver’s own work on the quartic surrogate is tracked separately, since evaluating the true function and its derivative is considerably more expensive than evaluating the model. Because we are solving a quartic (rather than a quadratic) problem, we use a multistart method to search for a global optimum. At each outer iteration k , 100 random points are drawn from a ball $B(0, \max\{1, d_{k-1}\})$. Using the default multistart setting for every method would be impractical across all 34 problems in this table.

Table 3: Algorithm ?? (PSB), varying only `model_solver`, on the 34 MGH problems.

Method	f	Converged (gradient)	Iterations	Func. evals	Grad. evals
Algorithm ?? (PSB, Newton)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	31/34	15.12 ± 21.90	30.21 ± 49.59	16.12 ± 21.90
Algorithm ?? (PSB, BFGS)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	31/34	15.26 ± 21.99	30.26 ± 48.63	16.26 ± 21.99
Algorithm ?? (PSB, BOBYQA)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	30/34	39.74 ± 108.86	57.68 ± 127.48	40.74 ± 108.86

Newton and BFGS are essentially indistinguishable (31/34, within 1% of each other on every column), and even BOBYQA, which needed an order of magnitude more evaluations and converged on only 10/34 when applied directly to the raw residual, falls behind by only one problem (30/34) and roughly $2.6\times$ more outer iterations on average. This distinction matters because evaluating the derivative can sometimes cost more than $2.6\times$ what evaluating the objective function costs. Since Newton achieves the best performance among the three, we adopt it as the model solver used throughout the rest of this work.

2.2 sum of square problem BFGS configuration

Now we will consider a different configuration than the average to BFGS method. We will make it, because the default linesearch, generally Hager Zhang linesearch, evaluate the derivative of function many times. For this reason, now we will test different classical linesearchs for BFGS method. The Table 4 checks some linesearchs for `Optim.BFGS` on the same 34 MGH problems with four line searches. Hager and Zhang (`Optim.jl`'s default), a minimalist backtracking with Armijo condition, and BackTracking with quadratic (order 2) and cubic (order 3) interpolation. "Converged (gradient)" counts how many of the 34 problems stopped under the gradient criterion. The remaining columns are mean $\pm$ standard deviation over all 34 problems (including the ones that stopped under a different criterion).

Table 4: Comparison of line searches for Optim.jl's BFGS on the 34 MGH problems.

Method	f	Converged (gradient)	Iterations	Func. evals	Grad. evals
Hager-Zhang	$1.29 \times 10^3 \pm 7.36 \times 10^3$	33/34	66.91 ± 120.92	107.82 ± 206.62	107.82 ± 206.62
Backtracking	$2.49 \times 10^7 \pm 1.45 \times 10^8$	30/34	51.56 ± 93.74	89.62 ± 131.90	52.47 ± 93.75
Backtracking (order 2)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	32/34	61.59 ± 99.26	86.74 ± 134.56	62.56 ± 99.17
Backtracking (order 3)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	31/34	56.91 ± 89.47	84.91 ± 132.01	57.85 ± 89.33

Hager-Zhang converges by gradient on the most problems (33/34) but at the highest average cost. the backtracking is a cheap linesearch but the value of f and congruence per problem on average isn't good (30/34). The backtracking order 2 and 3 have a near results. Indeed of Backtracking order 3 has a little less evaluations, the Backtracking order 2 found 32/34 gradients convergence. For this reason, we will chose Backtracking order 2 to compare with our approach.

2.2.1 Algorithm ?? against BFGS with backtracking

Algorithm ?? (PSB) uses persistent decaying σ , a full Hessian reset on a bad ratio, and $-\nabla f$ +Armijo at $k = 0$ and as a safeguard. The following table compares it against BFGS with backtracking line search on the 34 MGH least-squares problems.

Table 5: Algorithm ?? (PSB) vs. BFGS (backtracking): per-problem comparison on the MGH problems.

Problem	Method	f	$\ \nabla f\ $	Func. evals	Grad. evals	Criterion
MGH01	Algorithm ?? (PSB)	2.42×10^{-22}	9.82×10^{-12}	12	8	grad_conv
MGH01	BFGS (backtracking)	1.98×10^{-22}	3.69×10^{-10}	53	40	grad_conv
MGH02	Algorithm ?? (PSB)	5.10×10^{-18}	3.84×10^{-9}	10	7	grad_conv
MGH02	BFGS (backtracking)	1.11×10^{-26}	2.20×10^{-12}	15	11	grad_conv
MGH03	Algorithm ?? (PSB)	1.58×10^{-11}	2.57×10^{-9}	31	20	grad_conv
MGH03	BFGS (backtracking)	5.05×10^{-30}	2.83×10^{-10}	233	174	grad_conv
MGH04	Algorithm ?? (PSB)	3.81×10^{-17}	8.73×10^{-9}	10	8	grad_conv
MGH04	BFGS (backtracking)	9.86×10^{-32}	4.44×10^{-10}	40	14	grad_conv
MGH05	Algorithm ?? (PSB)	1.31×10^{-16}	6.95×10^{-9}	10	8	grad_conv
MGH05	BFGS (backtracking)	2.50×10^{-22}	1.08×10^{-10}	22	19	grad_conv
MGH06	Algorithm ?? (PSB)	1.01×10^3	1.16×10^{-28}	11	2	grad_conv
MGH06	BFGS (backtracking)	1.01×10^3	1.16×10^{-28}	10	2	grad_conv
MGH07	Algorithm ?? (PSB)	1.49×10^{-17}	6.03×10^{-9}	17	10	grad_conv
MGH07	BFGS (backtracking)	5.24×10^{-19}	5.03×10^{-9}	41	30	grad_conv
MGH08	Algorithm ?? (PSB)	4.11×10^{-3}	4.90×10^{-10}	12	9	grad_conv
MGH08	BFGS (backtracking)	4.11×10^{-3}	7.39×10^{-10}	34	28	grad_conv
MGH09	Algorithm ?? (PSB)	5.64×10^{-9}	7.55×10^{-11}	9	7	grad_conv
MGH09	BFGS (backtracking)	5.64×10^{-9}	1.55×10^{-9}	6	5	grad_conv
MGH10	Algorithm ?? (PSB)	4.40×10^1	1.20×10^{-5}	276	122	reg_stalled
MGH10	BFGS (backtracking)	4.40×10^1	2.28	552	386	step_conv

continued on next page

Table 5: (continued)

Problem	Method	f	$\ \nabla f\ $	Func. evals	Grad. evals	Criterion
MGH11	Algorithm ?? (PSB)	5.96×10^{-21}	3.25×10^{-13}	26	17	grad_conv
MGH11	BFGS (backtracking)	2.27×10^{-20}	7.17×10^{-10}	50	42	grad_conv
MGH12	Algorithm ?? (PSB)	3.80×10^{-18}	3.81×10^{-10}	14	11	grad_conv
MGH12	BFGS (backtracking)	1.58×10^{-21}	3.20×10^{-11}	53	41	grad_conv
MGH13	Algorithm ?? (PSB)	5.16×10^{-12}	9.61×10^{-9}	12	9	grad_conv
MGH13	BFGS (backtracking)	1.57×10^{-14}	3.61×10^{-9}	55	45	grad_conv
MGH14	Algorithm ?? (PSB)	9.42×10^{-19}	8.23×10^{-10}	11	7	grad_conv
MGH14	BFGS (backtracking)	3.52×10^{-22}	5.46×10^{-10}	44	31	grad_conv
MGH15	Algorithm ?? (PSB)	1.54×10^{-4}	9.06×10^{-9}	38	22	grad_conv
MGH15	BFGS (backtracking)	1.54×10^{-4}	4.13×10^{-10}	39	33	grad_conv
MGH16	Algorithm ?? (PSB)	4.29×10^4	6.53×10^{-5}	91	37	reg_stalled
MGH16	BFGS (backtracking)	4.29×10^4	2.83×10^{-5}	72	26	func_conv
MGH17	Algorithm ?? (PSB)	2.73×10^{-5}	8.38×10^{-9}	24	12	grad_conv
MGH17	BFGS (backtracking)	2.73×10^{-5}	8.23×10^{-9}	90	59	grad_conv
MGH18	Algorithm ?? (PSB)	1.00×10^{-18}	1.65×10^{-11}	58	34	grad_conv
MGH18	BFGS (backtracking)	2.83×10^{-3}	1.28×10^{-8}	44	41	grad_conv
MGH19	Algorithm ?? (PSB)	4.38×10^{-2}	9.20×10^{-9}	115	59	grad_conv
MGH19	BFGS (backtracking)	1.37×10^{-1}	8.29×10^{-10}	147	113	grad_conv
MGH20	Algorithm ?? (PSB)	1.14×10^{-3}	1.84×10^{-9}	13	10	grad_conv
MGH20	BFGS (backtracking)	1.14×10^{-3}	7.07×10^{-9}	49	39	grad_conv
MGH21	Algorithm ?? (PSB)	2.42×10^{-21}	3.11×10^{-11}	12	8	grad_conv
MGH21	BFGS (backtracking)	4.79×10^{-18}	1.50×10^{-9}	228	150	grad_conv
MGH22	Algorithm ?? (PSB)	4.52×10^{-14}	2.80×10^{-10}	13	10	grad_conv
MGH22	BFGS (backtracking)	1.65×10^{-13}	1.56×10^{-8}	136	112	grad_conv
MGH23	Algorithm ?? (PSB)	1.12×10^{-5}	2.28×10^{-9}	13	10	grad_conv
MGH23	BFGS (backtracking)	1.12×10^{-5}	1.21×10^{-8}	87	68	grad_conv
MGH24	Algorithm ?? (PSB)	1.73×10^{-6}	1.74×10^{-9}	37	22	grad_conv
MGH24	BFGS (backtracking)	1.73×10^{-6}	6.29×10^{-9}	588	450	grad_conv
MGH25	Algorithm ?? (PSB)	7.40×10^{-32}	3.85×10^{-16}	13	6	grad_conv
MGH25	BFGS (backtracking)	6.84×10^{-26}	7.27×10^{-12}	23	16	grad_conv
MGH26	Algorithm ?? (PSB)	1.40×10^{-5}	3.00×10^{-9}	27	16	grad_conv
MGH26	BFGS (backtracking)	1.40×10^{-5}	1.11×10^{-8}	36	34	grad_conv
MGH27	Algorithm ?? (PSB)	2.24×10^{-16}	1.94×10^{-9}	11	8	grad_conv
MGH27	BFGS (backtracking)	5.73×10^{-20}	3.69×10^{-9}	20	17	grad_conv
MGH28	Algorithm ?? (PSB)	2.22×10^{-15}	6.93×10^{-9}	10	8	grad_conv
MGH28	BFGS (backtracking)	2.53×10^{-16}	1.09×10^{-8}	26	18	grad_conv
MGH29	Algorithm ?? (PSB)	1.13×10^{-22}	1.91×10^{-11}	8	7	grad_conv
MGH29	BFGS (backtracking)	9.10×10^{-19}	1.56×10^{-9}	8	8	grad_conv
MGH30	Algorithm ?? (PSB)	7.64×10^{-27}	3.46×10^{-13}	10	7	grad_conv
MGH30	BFGS (backtracking)	1.97×10^{-19}	4.03×10^{-9}	46	23	grad_conv
MGH31	Algorithm ?? (PSB)	1.53	4.17×10^{-8}	49	18	reg_stalled
MGH31	BFGS (backtracking)	1.34	4.48×10^{-9}	78	43	grad_conv
MGH32	Algorithm ?? (PSB)	5.00	3.85×10^{-16}	3	2	grad_conv
MGH32	BFGS (backtracking)	5.00	5.58×10^{-16}	2	2	grad_conv
MGH33	Algorithm ?? (PSB)	2.32	6.33×10^{-11}	12	5	grad_conv
MGH33	BFGS (backtracking)	2.32	1.42×10^{-10}	13	5	grad_conv
MGH34	Algorithm ?? (PSB)	3.07	1.55×10^{-10}	9	2	grad_conv
MGH34	BFGS (backtracking)	3.07	2.13×10^{-10}	9	2	grad_conv

Across the 34 problems, the two methods usually agree closely on the final objective value, but differ

sharply in cost. Algorithm ?? (PSB) needs far fewer function and gradient evaluations than BFGS on most problems (see Table 5). MGH10 shows how hard some of these problems are. Neither method certifies gradient convergence there, and both land on essentially the same objective value ($f \approx 43.97$), yet Algorithm ?? still reaches a far smaller gradient norm than BFGS ($\|\nabla f\| \approx 1.2 \times 10^{-5}$ against ≈ 2.28), using half the function evaluations and under a third of the gradient evaluations. MGH18 shows the opposite kind of gap. Although both methods stop under the gradient criterion, Algorithm ?? reaches essentially zero ($f \approx 1.0 \times 10^{-18}$) while BFGS settles for a distinctly larger value ($f \approx 2.8 \times 10^{-3}$). A small gradient norm does not always mean the same stationary point.

The boxplot below compares the two approaches directly and uses the Mann-Whitney U test to check whether the difference between them is statistically significant.

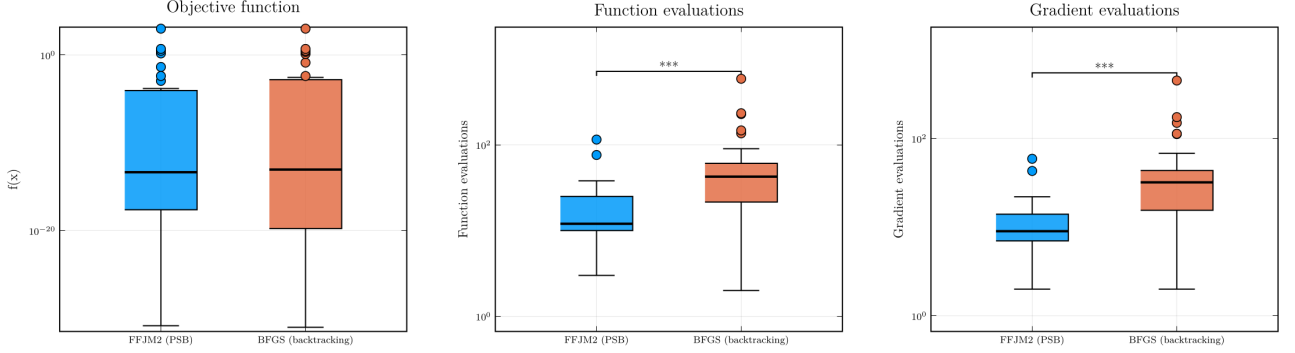


Figure 2: Comparison of Algorithm ?? (PSB) vs. BFGS (backtracking) on the MGH problems, restricted to runs where both methods converged by gradient norm. Final f value, function evaluations, and gradient evaluations. Stars indicate significance under the Mann-Whitney U test [7] ($p < 0.05$: *; $p < 0.01$: **; $p < 0.001$: ***).

As Figure 2 shows, the objective function values are not very different between the two methods. The p-value confirms neither achieves a significantly better result. Function evaluations and gradient evaluations, however, are different, and the p-value confirms this difference is significant. Algorithm ?? therefore performs better than BFGS in this respect, reaching a comparable solution at a much lower cost.

The boxplot above is restricted to the subset of problems where both methods converge by gradient norm. To compare the two methods over all 34 problems, Figure 3 shows the performance profile of Dolan and Moré [4] for the final objective value, function evaluations, and gradient evaluations. For each problem, a method's ratio r is its evaluation count divided by the smaller of the two methods' counts on that same problem, so the faster method has $r = 1$. A run that does not converge by gradient norm counts as a failure, with $r = \infty$ (a problem on which neither method converges by gradient norm counts as a mutual failure, $r = \infty$ for both, still included in the 34). The curve $\rho_s(\tau)$ is the fraction of the 34 problems on which method s 's ratio is at most τ , plotted against $\log_2 \tau$: $\rho_s(1)$ is therefore the fraction of problems on which s is the faster (or tied) method, and $\rho_s(\tau)$ as $\tau \rightarrow \infty$ approaches s 's overall convergence rate, independent of cost.

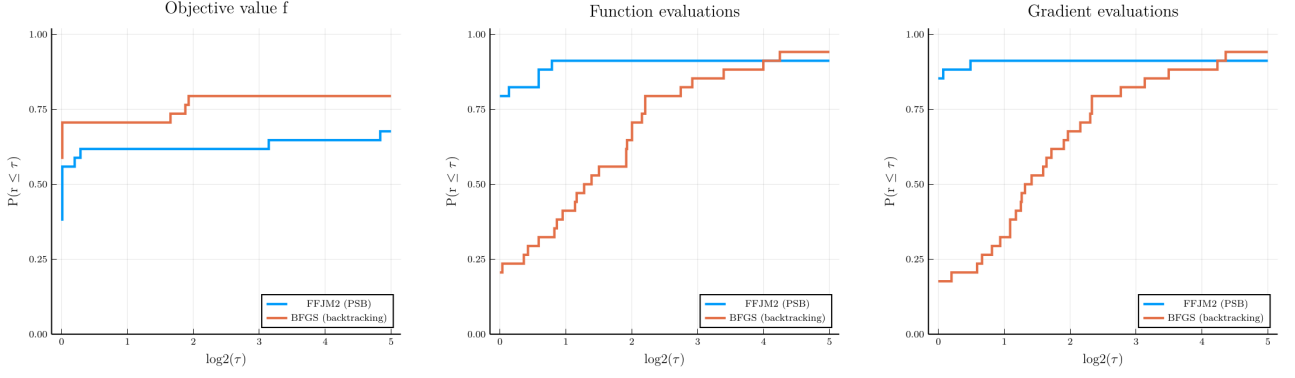


Figure 3: Performance profile [4] of Algorithm ?? (PSB) vs. BFGS (backtracking) on all 34 MGH problems, for the final objective value (left), function evaluations (middle), and gradient evaluations (right). $\rho_s(\tau)$ is the fraction of problems on which method s 's cost (final f value, function evaluations, or gradient evaluations, matching the panel) is at most τ times the smaller of the two methods' costs on that problem, plotted against $\log_2 \tau$. A run that does not converge by gradient norm counts as a failure.

Looking only at the objective value in Figure 3, BFGS comes out ahead, the better (or tied) method on close to 59% of the 34 problems against 38% for Algorithm ?. Table 5 and Figure 2, however, already showed that this difference in objective value is not statistically significant. The Mann-Whitney test does not treat it as a real gap in solution quality between the two methods. For function and gradient evaluations, by contrast, Algorithm ? is the better (or tied) method on about 80% of the 34 problems, and that difference *is* significant. Taken together, these results make Algorithm ? an attractive alternative to BFGS. It gives up no solution quality worth mentioning, while reaching that solution at a significantly lower cost.

3 Coefficient estimation problem

3.1 The Saint-Venant model and the calibration problem

The Saint-Venant equations [2, 6, 11, 12] describe the flow of fluids in natural channels. The state variables that describe a natural channel vary with respect to $x \in [x_{min}, x_{max}]$ (position, usually measured in meters) and $t \in [t_{min}, t_{max}]$ (time, usually measured in seconds). We define:

- $z(x, t)$: surface elevation
- $h(x, t) = z(x, t) - z_b(x)$: river depth at (x, t)
- $A(x, t)$: cross-sectional wetted area
- $Q(x, t)$: flow rate
- $P(x, t)$: wetted perimeter
- $R(x, t) = A(x, t)/P(x, t)$: hydraulic radius
- g : acceleration of gravity (approximately $9.81m/s^2$)

The Saint-Venant equations consist of two main components: Equation (4) describes mass conservation and equation (5) represents conservation of the linear momentum.

$$\frac{\partial A}{\partial t} + \frac{\partial Q}{\partial x} = 0 \quad (4)$$

and

$$\frac{\partial Q}{\partial t} + \frac{\partial}{\partial x} \left(\frac{Q^2}{A} \right) + gA \frac{\partial z}{\partial x} + \frac{n_g^2 Q |Q|}{AR^{4/3}} = 0. \quad (5)$$

The Manning (also called Yen-Manning [1]) coefficient n_g represents roughness [1]. Typically, given the inlet discharge $Q(x_{min}, t)$ for $t \in [t_{min}, t_{max}]$, initial conditions $A(x, t_{min})$ and $Q(x, t_{min})$, and the coefficient n_g , numerical methods compute the values of $Q(x, t)$ and $A(x, t)$ at a set of discrete “grid points” (x_ν, t_ν) . This PDE system is solved using an explicit finite difference method with artificial diffusion of 0.99, following [6, 11]. Unfortunately, the Yen-Manning coefficient n_g is often unknown for natural channels and can vary with position x and even time t . For more details, see [1].

Following [5], this research assumes that the Manning coefficient n_g varies with position x , and that a set of elevation observations $z_{obs}(x_\nu, t_\nu)$ are available at n_{obs} distinct locations and times, indexed by $\nu = 1, \dots, n_{obs}$. Consequently, the spatial distribution of $n_g(x)$, discretized over a suitable grid, becomes an unknown component of our problem. Furthermore, to ensure physically plausible results, minimal variation will be required for a function $n_g(x)$ that minimizes spatial oscillations while fitting the solution of the Saint-Venant equations to the observations.

Consider a grid with n_{grid} points $x_{min} = x_1 < \dots < x_{n_{grid}} = x_{max}$. Collect the n_{grid} values $n_g(x_1), \dots, n_g(x_{n_{grid}})$ into a parameter vector $\xi \in \mathbb{R}^{n_{grid}}$, with $\xi_i := n_g(x_i)$ for $i = 1, \dots, n_{grid}$, kept notationally distinct from the spatial coordinate x . In [5], this calibration problem is posed as a constrained optimization problem, minimizing either the sum of squared deviations of ξ from a reference value $\bar{n}$ or the sum of absolute deviations, subject to equality constraints requiring the simulated elevations to match the n_{obs} observations and a box constraint $0 \leq \xi_i \leq n_g^{\max}$. It is solved there by a Safeguarded Augmented Lagrangian method (Algorithm 2.1 of [5]), with BOBYQA or PRIMA handling the subproblems.

The two applications differ mainly in how the underlying PDE is solved. In this work we rely on a more stable PDE solver, which lets us compute the derivative of the objective with respect to ξ accurately. This is precisely what enables the gradient-based comparison carried out below, namely BFGS and Algorithm ?? against the derivative-free BOBYQA. Figure 4 contrasts the previous and the current solver on the same problem.

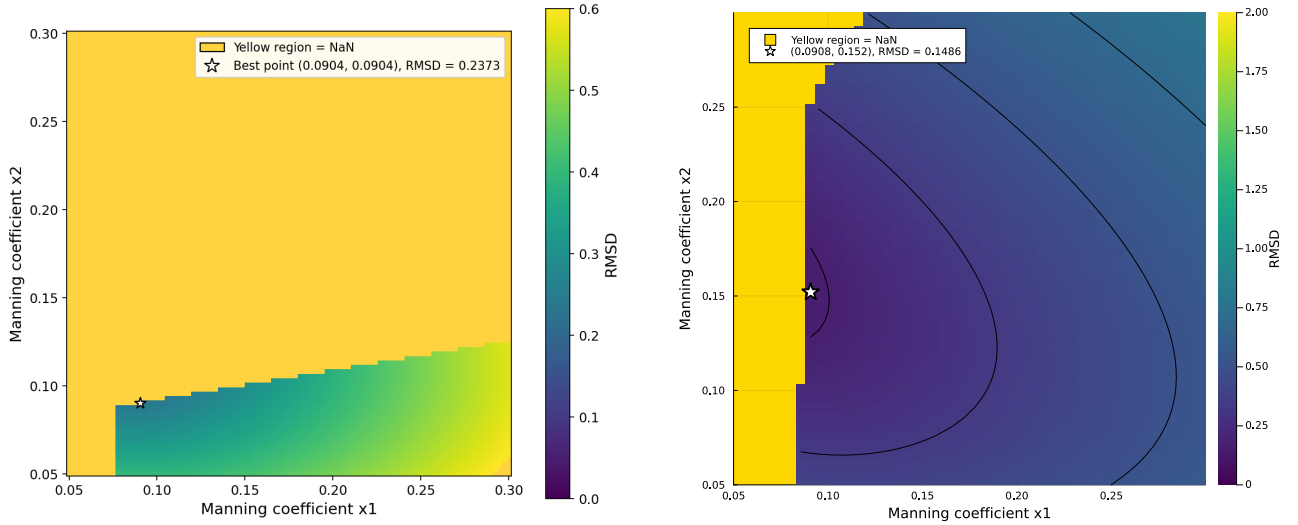


Figure 4: Comparison between the previous (left) and the current (right) PDE solver.

For the comparison in this section, we adapt this into the unconstrained nonlinear least-squares form used throughout this report, minimizing the elevation mismatch directly instead of enforcing it as an equality constraint. For $i = 1, \dots, m$ (with $m = n_{obs}$), let $z_i(\xi)$ denote the surface elevation the Saint-Venant solution predicts, for parameter vector ξ , at the i -th observation’s location and time (x_i, t_i) , and let $z_{obs,i} = z_{obs}(x_i, t_i)$ be the corresponding observation. Define the residual components $f_i(\xi) = z_i(\xi) - z_{obs,i}$, collect them into $F(\xi) = (f_1(\xi), \dots, f_m(\xi))^T$, and minimize

$$f(\xi) = \frac{1}{2} \|F(\xi)\|_2^2 = \frac{1}{2} \sum_{i=1}^m f_i(\xi)^2.$$

Calibrating n_g against the observations is thus the nonlinear least-squares problem of finding ξ that

minimizes $f(\xi)$ (ideally $F(\xi) = 0$, though the real data need not be exactly consistent with the model).

3.2 Experimental setup

To calibrate ξ against this residual, four benchmark scenarios compare the same three solvers used throughout this report, namely BFGS (backtracking, order 2), BOBYQA, and Algorithm ??, applied directly to $F(\xi)$ above where each solver stands in as the whole optimizer, rather than as Algorithm ??’s internal subproblem solver. Two scenarios use pregenerated data, where the observations $z_{obs,i}$ are themselves generated by the model at a known ξ^* , so the true minimizer is known exactly. The other two use the real observed data, where the true minimizer (if it exists) is unknown. BFGS and BOBYQA are both given the same box, $[0, 0.5]^{n_{grid}}$, and for the same reason, since a trial step whose direction is too large is likely to land in a forbidden region P , in the terminology of [5], this algorithm’s own reference, a region where the PDE simulation diverges and the residual is undefined, so evaluating F there is meaningless and wasted. BFGS enforces the box softly, through an additive penalty term, $\frac{1}{2}\|F(\xi)\|_2^2 + \text{penalty}(\xi)$, where penalty (weight 10^6) evaluates to exactly 0 whenever every coordinate of ξ is feasible, as is the case at every minimizer reported below. F is still evaluated everywhere, but the penalty discourages the line search from accepting a step that reaches too far. BOBYQA instead enforces the same box as a hard constraint, through NLOpt’s own bound support, so it minimizes $\frac{1}{2}\|F(\xi)\|_2^2$ unpenalized and NLOpt never evaluates the objective at a trial point outside the box in the first place. Algorithm ?? minimizes $f(\xi) = \frac{1}{2}\|F(\xi)\|_2^2$ directly, with no box at all, relying instead on its own σ -regularization to keep steps away from that same region.

3.2.1 Algorithmic settings

All three solvers are implemented in Julia, version 1.10, and every experiment reported in this section was run on a computer with a 5.2 GHz 12th Gen Intel(R) Core(TM) i9-12900K processor and 16 GB of 3200 MHz DDR4 RAM, running Linux Mint 21.1. They also share the same evaluation budget, since each residual evaluation is a full transient simulation of the Saint-Venant equations (BFGS and BOBYQA allowed at most 1000 evaluations of the residual, Algorithm ?? at most 1000 outer iterations), and BFGS and Algorithm ?? further share a gradient-norm convergence tolerance of 10^{-3} ; the remaining settings differ per solver, as detailed below. The Status column in Tables 6, 7, 8, and 9 reports each solver’s own termination code directly, `grad_conv` (Algorithm ?? or BFGS) and `Trust_Region` (BOBYQA) marking a stopping criterion that actually certifies stationarity, `reg_stalled` (Algorithm ??’s σ stuck at its ceiling) and `step_conv` (BFGS’s line search stuck at its minimum step) marking a stall instead, and `maximum_iterations`, `XTOL_REACHED`, and `FTOL_REACHED` marking the budget above running out without either.

- **BFGS**, the BFGS method from the `Optim.jl` package (version 2.0), together with the order-2 backtracking line search from the `LineSearches.jl` package (version 7.7), the same combination used in Section 2.1. Gradients are obtained by forward-mode automatic differentiation with the `ForwardDiff.jl` package (version 1.3), as discussed above. The line search gives up, reporting `step_conv`, once its step length shrinks below 10^{-12} , and the number of gradient evaluations is capped at 500.
- **BOBYQA**, the implementation provided by the `NLOpt.jl` package (version 1.2). The trust region starts with radius 0.01 and the search stops once it has shrunk to 10^{-6} . The number of interpolation points and other internal settings are left at the package’s own defaults.
- **Algorithm ??**, our own Julia implementation, like the Saint-Venant solver itself. It uses the same regularization schedule described in Section 2.1, starting at $\sigma_0 = 1$ and growing by a factor of $\sigma_{grow} = 10$ on every rejected step, with a floor of $\sigma_{min} = 10^{-8}$ and a ceiling of $\sigma_{max} = 10^{12}$ kept purely for numerical stability.

3.3 Results

Table 6: Pregenerated-data experiment, $n_{grid} = 2$: $\xi^* = (0.20, 0.15)$, $\xi^0 = (0.25, 0.25)$.

Method	RMSD	f	$\ \nabla f\ $	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	7.47×10^{-7}	9.44×10^{-11}	1.48×10^{-3}	16	9	384.2	grad_conv
BOBYQA	4.61×10^{-8}	3.59×10^{-13}	6.94×10^{-5}	30	0	326.3	XTOL_REACHED
Algorithm ??	5.86×10^{-11}	5.80×10^{-19}	6.65×10^{-8}	10	5	158.0	grad_conv

All three methods converge to essentially the exact reference $\xi^* = (0.20, 0.15)$. Algorithm ?? is the cheapest of the three by every cost measure (fewest evaluations, least time) despite also reaching the tightest gradient norm and RMSD.

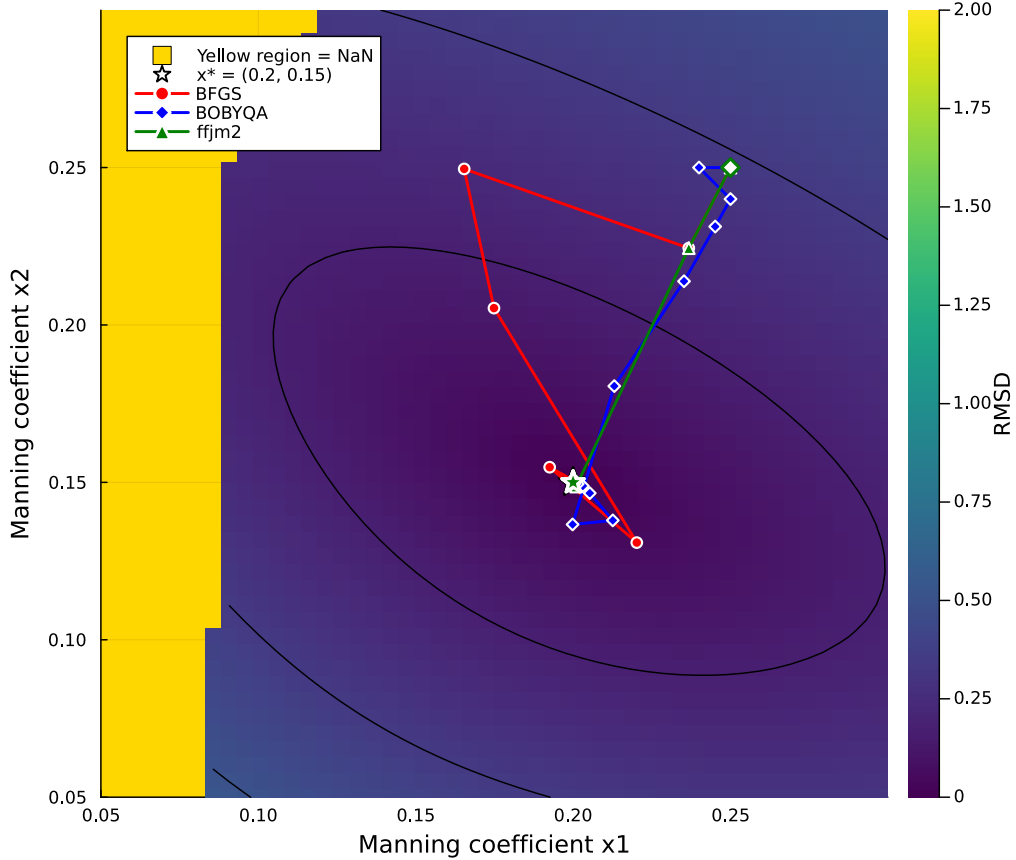


Figure 5: RMSD heatmap of the pregenerated-data experiment ($n_{grid} = 2$) with the path of accepted iterates of each method overlaid: BFGS (red), BOBYQA (blue), and Algorithm ?? (green). The white diamond marks the shared starting point $\xi^0 = (0.25, 0.25)$. The star marks each method's final iterate.

Table 7: Pregenerated-data experiment, $n_{grid} = 10$: decreasing profile ξ^* from 0.20 to 0.15, $\xi^0 = (0.25, \dots, 0.25)$.

Method	RMSD	f	$\ \nabla f\ $	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	2.00×10^{-1}	6.73×10^0	3.33×10^2	166	28	2453.6	step_conv
BOBYQA	3.68×10^{-4}	2.29×10^{-5}	1.26×10^{-1}	1000	0	12484.1	MAXEVAL_REACHED
Algorithm ??	4.38×10^{-6}	3.24×10^{-9}	1.02×10^{-3}	15	9	444.3	grad_conv

At $n_{grid} = 10$ Algorithm ?? is now the clear winner. It is the only method that certifies a genuine stopping criterion (grad_conv, $\|\nabla f\| \approx 1.02 \times 10^{-3}$), and it does so at a small fraction of the cost of the other two (15 function and 9 gradient evaluations, 444.3s). BFGS's line search stalls (step_conv with the worst RMSD and gradient norm of the three (2.00×10^{-1} and $\approx 3.33 \times 10^2$, respectively). BOBYQA gets much closer (RMSD $\approx 3.68 \times 10^{-4}$, $\|\nabla f\| \approx 1.26 \times 10^{-1}$) but exhausts its evaluation

budget (`MAXEVAL_REACHED`, 1000 function evaluations) before certifying convergence. At $n_{grid} = 2$ all three methods converged, with Algorithm ?? already the cheapest of the three.

Table 8: Real data, $n_{grid} = 2$: $\xi^0 = (0.25, 0.25)$.

Method	RMSD	f	$\ \nabla f\ $	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	2.10×10^{-1}	7.44×10^0	5.31×10^3	228	32	3191.3	<code>step_conv</code>
BOBYQA	1.42×10^{-1}	3.41×10^0	1.65×10^2	67	0	722.7	<code>Trust_Region</code>
Algorithm ??	1.42×10^{-1}	3.42×10^0	1.80×10^2	53	21	806.8	<code>reg_stalled</code>

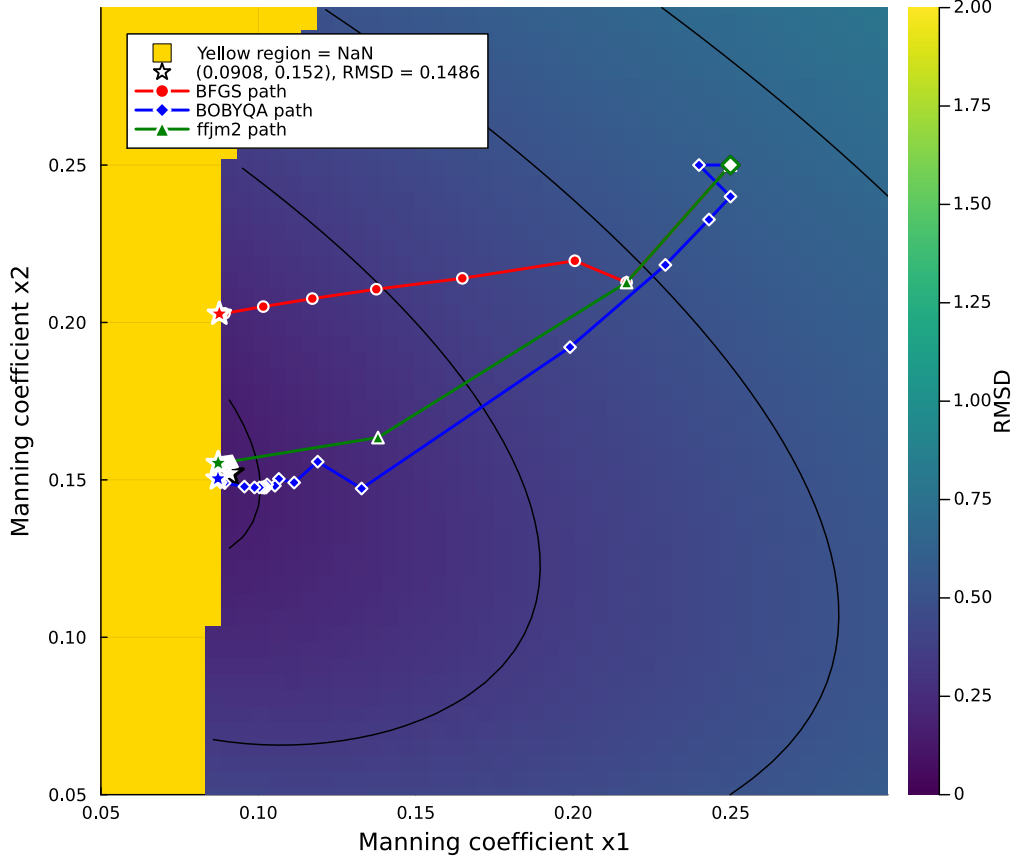


Figure 6: RMSD heatmap of the real-data experiment ($n_{grid} = 2$) with the path of accepted iterates of each method overlaid: BFGS (red), BOBYQA (blue), and Algorithm ?? (green). The white diamond marks the shared starting point $\xi^0 = (0.25, 0.25)$, the star marks each method’s final iterate.

On real data, BFGS’s line search stalls (`step_conv`) before reaching a good point, with the worst RMSD of the three (2.10×10^{-1}) and a gradient norm still at $\approx 5.31 \times 10^3$. BOBYQA and Algorithm ?? land on essentially the same minimizer instead ($\xi \approx (0.087, 0.155)$), with matching RMSD (1.42×10^{-1}) and $f \approx 3.41$ – 3.42 . BOBYQA certifies its own `Trust_Region` criterion using fewer total evaluations and less wall-clock time (722.7s versus 806.8s), while Algorithm ?? again stalls on σ (`reg_stalled`) just short of that same point.

Table 9: Real data, $n_{grid} = 10$: $\xi^0 = (0.25, \dots, 0.25)$.

Method	RMSD	f	$\ \nabla f\ $	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	5.15×10^{-2}	4.49×10^{-1}	1.26×10^2	221	61	6253.5	<code>step_conv</code>
BOBYQA	4.68×10^{-2}	3.70×10^{-1}	8.67×10^{-1}	772	0	9120.5	<code>Trust_Region</code>
Algorithm ??	4.58×10^{-2}	3.54×10^{-1}	7.08×10^{-1}	60	24	1258.2	<code>reg_stalled</code>

This remains the hardest of the four scenarios: none of the three methods certifies a genuine stopping

criterion. BFGS’s line search stalls (`step_conv`) with the largest gradient norm of the three (1.26×10^2) and the worst RMSD (5.15×10^{-2}). BOBYQA certifies its own `Trust_Region` criterion, using 772 evaluations and 2.5h to reach RMSD near to 4.68×10^{-2} . Algorithm ??’s regularization parameter again hits its ceiling (`reg_stalled`), but this time it is the best of the three by every measure: the smallest gradient norm (7.08×10^{-1} , even below BOBYQA’s), the lowest RMSD (4.58×10^{-2}), and by far the lowest cost (60 residual and 24 Jacobian evaluations, 21min, against BOBYQA’s 772 evaluations and BFGS’s 282). This is the opposite of the $n_{grid} = 10$ pregenerated-data scenario (Table 7), where the same stall left Algorithm ?? the worst of the three – here it is both the cheapest and the most accurate. All three still land on different minimizers with comparable RMSD ($4.6\text{--}5.2 \times 10^{-2}$), and none certifies first-order optimality.

4 Conclusion

This paper describes an approach based on quartic residual models for solving sum-of-squares problems whose objective function, and especially its gradient, is expensive to evaluate, and whose domain contains forbidden regions where the objective is undefined, unreliable, or has no physical meaning, without the boundary of those regions being known in advance.

Section 1 addresses a degeneracy at the algorithm’s very first iteration ($k = 0$), where no curvature information is yet available. The quartic subproblem collapses to a damped Gauss-Newton step there, which we handle with a steepest-descent, Armijo-line-search safeguard.

Section 2 uses the 34 MGH sum-of-squares problems to calibrate Algorithm ??, choosing the PSB Hessian-update model and Newton as the quartic-subproblem solver for their lower cost, and an order-2 backtracking line search for the BFGS baseline. Compared on all 34 problems, the two methods reach statistically indistinguishable objective values (Mann-Whitney test), but Algorithm ?? is significantly cheaper, the better or tied method on about 80% of the problems.

Section 3 applies all three methods to a real engineering problem, calibrating the Manning coefficient in the Saint-Venant equations, across four scenarios (synthetic and real data, dimensions 2 and 10). Algorithm ?? is the cheapest and most accurate of the three in three of the four scenarios; the exception is the synthetic dimension-10 case, where its regularization parameter σ stalls and it becomes the worst of the three. The same failure mode reappears, without the same cost, in the hardest scenario, real data at dimension 10, where Algorithm ?? is once again the cheapest and most accurate of the three.

Taken together, these results suggest that the quartic-model approach is a competitive alternative to BFGS and BOBYQA for expensive sum-of-squares problems with forbidden regions, typically cheaper whenever it converges, though its regularization-stall failure mode, first seen in the dimension-10 experiments, remains an open point for future work.

References

- [1] E. G. Birgin, M. R. Correa, V. A. González-López, J. M. Martínez, and D. S. Rodrigues, *Randomly supported variation of deterministic models and its application to one-dimensional shallow water flows*, J. Hydraulic Eng., 150 (2024), 04024026.
- [2] O. Castro-Orgaz and W. H. Hager, *Shallow Water Hydraulics*, Springer, 2019.
- [3] J. E. Dennis, Jr. and H. Wolkowicz, *Sizing and least-change secant methods*, SIAM J. Numer. Anal., 30 (1993), pp. 1291–1314.
- [4] E. D. Dolan and J. J. Moré, *Benchmarking optimization software with performance profiles*, Math. Program., 91 (2002), pp. 201–213.
- [5] F. A. Fortunato and J. M. Martínez, *KKT-free Constrained Optimization and Application to River-Flow Modeling*, IMECC, University of Campinas, preprint, 2025.
- [6] R. J. LeVeque, *Numerical Methods for Conservation Laws*, Lectures in Mathematics, ETH Zürich, Birkhäuser, 1992.
- [7] H. B. Mann and D. R. Whitney, *On a test of whether one of two random variables is stochastically larger than the other*, Ann. Math. Statist., 18 (1947), pp. 50–60.
- [8] P. K. Mogensen and A. N. Riseth, *Optim: A mathematical optimization package for Julia*, J. Open Source Software, 3 (2018), p. 615.
- [9] J. J. Moré, B. S. Garbow, and K. E. Hillstom, *Testing unconstrained optimization software*, ACM Trans. Math. Software, 7 (1981), pp. 17–41.
- [10] M. J. D. Powell, *A new algorithm for unconstrained optimization*, in Nonlinear Programming, J. B. Rosen, O. L. Mangasarian, and K. Ritter, eds., Academic Press, New York, 1970, pp. 31–65.
- [11] R. M. Porto, *Hidráulica Básica*, EESC-USP, São Paulo, 2000.
- [12] A. J. C. Saint-Venant, *Théorie du mouvement non-permanent des eaux, avec application aux crues des rivières et à l'introduction des marées dans leur lit*, Comptes Rendus des Séances de Académie des Sciences, 73 (1871), pp. 147–154.